Accelerating Event-Camera Preprocessing: From a Threading Question to a Targeted Numba Fix

A case study in diagnosing a data-pipeline bottleneck, testing the wrong fix first, and converging on the right one.

---

1. The Initial Question

The pipeline feeds the GPU using PyTorch's standard multiprocessing DataLoader workers: separate OS processes fetch and preprocess raw event-camera data (denoise, then convert to dense frames) in parallel while the GPU trains on the previous batch. The question that started this investigation was whether that architecture could be replaced with a lighter, thread-based worker abstraction instead -- and whether that would be faster.

The specific concern was DSEC, a 640×480 event-camera dataset -- by far the highest-resolution dataset this pipeline supports, and one where per-sample preprocessing cost was suspected to be a real bottleneck, severe enough that even a batch size of 1 might strain available GPU memory.

2. Investigation: Does the Preprocessing Chain Even Support Threading?

Python's Global Interpreter Lock (GIL) means a pure-Python thread pool cannot run two pieces of Python code at the same time on separate CPU cores -- unless the specific code running releases the GIL. So the first real question was empirical, not architectural: **does this pipeline's preprocessing code actually release the GIL at all?**

Two preprocessing stages exist per sample: **denoising** (a numba-JIT-compiled function, `_denoise_core_numba`) and **framing** (tonic's own `ToFrame`, converting raw events into dense per-timestep tensors). A dedicated probe (`diagnostics/probe_preprocessing_gil_release.py`) measured GIL release directly: time a CPU-bound Python workload alone, time the target function alone, then run both concurrently on separate threads. If the target releases the GIL, the concurrent run finishes close to the slower of the two individual times (real overlap). If it holds the GIL, the concurrent run takes close to the sum of both (no overlap at all).

Target	Overlap ratio (Windows)	Overlap ratio (WSL/Linux)	Verdict
denoise, as shipped (no nogil)	1.01 - 1.12	1.12	GIL held
denoise, with nogil=True added	1.36	1.56	GIL released
tonic ToFrame (unmodified)	1.02 - 1.08	1.23	GIL held

This confirmed two things. First, the denoise kernel *can* release the GIL -- it just isn't configured to, because numba's `@njit` decorator does not release the GIL by default; the `nogil=True` flag has to be requested explicitly, and the function's body (pure numeric array operations, no Python objects) makes it a safe candidate for that flag. Second, and more importantly: **tonic's ToFrame does not release the GIL, and per-call timing showed it is also the more expensive of the two stages** -- roughly 40-66 ms per sample against denoise's ~10 ms.

3. The Refined Question

This result reframed the original question. A thread-based worker pool can only ever parallelize the stage that actually releases the GIL. Making the cheaper stage (denoise) thread-friendly would not help, because the expensive stage (ToFrame) would still serialize on every thread regardless of how many threads exist. Process-based multiprocessing -- what the pipeline already uses -- sidesteps the GIL question entirely, because every worker is a separate OS process with its own interpreter. Switching to threads would therefore be a strictly worse architecture for this pipeline, not a faster one.

The question this investigation actually needed to answer was not *"how do we parallelize preprocessing across threads?"* but: **"what is the cheapest, lowest-risk way to reduce the real, dominant cost in the preprocessing chain -- regardless of threading model?"**

4. The Derived Solution

ToFrame's binning operation is, at its core, a simple accumulation: for every event, increment one cell of a (time-bin, polarity, y, x) output array. Addition is commutative -- unlike the denoise algorithm (a strict, order-dependent scan over a running per-pixel timestamp), ToFrame's accumulation has **no dependency on the order events are processed in**. That makes it a safe, well-understood target for the exact same recipe this codebase had already proven out once before: replace a slow, pure-Python/NumPy reference implementation with a numba-JIT-compiled kernel that runs the identical algorithm, verified byte-identical against the original on real data -- the same approach that produced `FastDenoise` as a ~9-72x faster, drop-in replacement for tonic's own event-denoising step.

This is real, bounded engineering effort -- not a one-line change -- but it is low-risk because it follows an established pattern already proven correct in this exact codebase, and it improves preprocessing cost for every dataset the pipeline supports, not only DSEC.

5. Implementation

`event_data_workflow/fast_to_frame.py` implements the replacement, scoped to the `n_time_bins` framing mode -- the pipeline's shipped default. It reproduces tonic's reference algorithm exactly: bin boundaries are located via binary search over the sorted event timestamps (matching NumPy's `searchsorted` "left" semantics precisely, including the reference's own floor-division truncation, which silently drops a handful of trailing events past the last bin), then a single linear pass over the events accumulates counts into the output tensor. Only the execution engine changes -- from interpreted Python/NumPy to compiled machine code -- not the algorithm's behaviour.

```
"""Fast replacement for tonic.functional.to_frame_numpy's n_time_bins path, which slices
events into T sub-arrays in pure Python/NumPy (np.add.at per slice) and was measured at
40-66ms/sample versus FastDenoise's ~10ms (diagnostics/probe_preprocessing_gil_release.py) --
the dominant cost in the per-sample preprocessing chain.

Scope: n_time_bins mode ONLY, matching data_workflow.yaml's actually-used default
(framing.mode). time_window mode is not covered -- its bin count is variable per
recording rather than fixed, a different algorithm tonic's slice_events_by_time
implements; extending this would need its own kernel and its own validation pass.

Same recipe as fast_denoise.py: JIT-compiled, same control flow as tonic's reference
(bin boundaries via binary search on sorted timestamps, then one linear pass over events
accumulating counts per (bin, polarity, y, x)) -- only the execution engine differs.
Unlike denoise's timestamp-scan, this accumulation has NO order dependency (addition is
commutative), so it is a safe target for future thread/parallel work; that is not
attempted here, matching fast_denoise.py's own single-threaded-first precedent.

Falls back to tonic's own to_frame_numpy if numba is unavailable -- correct but slower,
never a crash, same fallback contract as fast_denoise.py.
"""
import numpy as np

try:
    from numba import njit
    _NUMBA_AVAILABLE = True
except ImportError:
    _NUMBA_AVAILABLE = False

    def njit(*args, **kwargs):
        def decorator(fn):
            return fn
        return decorator


@njit(cache=True)
def _searchsorted_left(t, target):
    """First index where t[idx] >= target, matching np.searchsorted(t, target, side='left')
    on a sorted array. Written out rather than calling np.searchsorted directly so the
    exact 'left' semantics are guaranteed regardless of numba version support."""
    lo, hi = 0, t.shape[0]
    while lo < hi:
        mid = (lo + hi) // 2
        if t[mid] < target:
            lo = mid + 1
        else:
            hi = mid
    return lo


@njit(cache=True)
def _bin_boundaries_numba(t, n_time_bins):
    """T+1 boundary indices into the (sorted-by-time) events array -- boundary[i] is where
    bin i starts, boundary[T] is where the last bin ends. Matches
    SliceByTimeBins.get_slice_metadata with overlap=0 exactly: time_window = (t[-1]-t[0])
    // n_time_bins (integer floor division), stride == time_window, so consecutive bins
```

```
    tile the array with no gap and no overlap -- boundary[i+1] IS indices_end[i] IS
    indices_start[i+1]. Floor-division truncation means boundary[T] can be < len(t): a
    few trailing events past the last bin's end are silently dropped, matching the
    reference's own behavior exactly rather than "helpfully" keeping them."""
    n = t.shape[0]
    t0 = t[0]
    time_window = (t[n - 1] - t0) // n_time_bins
    boundaries = np.empty(n_time_bins + 1, dtype=np.int64)
    for i in range(n_time_bins + 1):
        boundaries[i] = _searchsorted_left(t, t0 + i * time_window)
    return boundaries


@njit(cache=True)
def _accumulate_numba(x, y, p, boundaries, n_time_bins, height, width, n_polarities):
    """One linear pass over events (only up to boundaries[-1] -- see the tail-drop note
    above), advancing the bin pointer as boundaries are crossed, incrementing
    frames[bin, p, y, x] per event -- the numba-compiled equivalent of tonic's per-slice
    np.add.at(frames, (i, p, y, x), 1)."""
    frames = np.zeros((n_time_bins, n_polarities, height, width), dtype=np.int16)
    end = boundaries[n_time_bins]
    bin_idx = 0
    for j in range(end):
        while bin_idx < n_time_bins - 1 and j >= boundaries[bin_idx + 1]:
            bin_idx += 1
        frames[bin_idx, p[j], y[j], x[j]] += 1
    return frames


def to_frame_numba(events, sensor_size, n_time_bins: int):
    """Drop-in for tonic.functional.to_frame_numpy(..., n_time_bins=n_time_bins) -- same
    (T, P, H, W) int16 output. Does not replicate the empty-events zero-fill branch;
    callers (FastToFrame below) must guard len(events) == 0 the same way
    tonic.transforms.ToFrame.__call__ does, before this is ever invoked."""
    if not _NUMBA_AVAILABLE:
        from tonic.functional import to_frame_numpy
        return to_frame_numpy(events, sensor_size=sensor_size, n_time_bins=n_time_bins)

    width, height, n_polarities = sensor_size
    x = events["x"].astype(np.int64)
    y = events["y"].astype(np.int64)
    p = events["p"].astype(np.int64)
    t = events["t"].astype(np.int64)

    boundaries = _bin_boundaries_numba(t, n_time_bins)
    return _accumulate_numba(x, y, p, boundaries, n_time_bins, height, width, n_polarities)


class FastToFrame:
    """Drop-in replacement for tonic.transforms.ToFrame (n_time_bins mode only) --
    a picklable class, not a closure, for Windows' spawn-based multiprocessing (same
    reason FastDenoise/FixedToFrame are classes, not functions)."""

    def __init__(self, sensor_size, n_time_bins: int):
        self.sensor_size = sensor_size
        self.n_time_bins = n_time_bins

    def __call__(self, events):
        if len(events) == 0:
            w, h, p = self.sensor_size
            return np.zeros((self.n_time_bins, p, h, w), dtype=np.int16)
        return to_frame_numba(events, sensor_size=self.sensor_size, n_time_bins=self.n_time_bins)
```

event_data_workflow/fast_to_frame.py -- full implementation. This is the actual deliverable of this investigation, and remains in the codebase.

6. Validation Results

Tested against every dataset registered in this pipeline that could be loaded without a pre-existing, unrelated infrastructure issue -- see the note below the table. Methodology and full code for how these numbers were produced are documented in Section 9.

6a. Correctness -- byte-identical against tonic's reference

Dataset	Samples compared	Byte-identical
N-MNIST	25	25 / 25
DVS128 Gesture	25	25 / 25
N-Caltech101	25	25 / 25
Eye Tracking	25	25 / 25
DSEC (single recording, thun_00_a)	10	10 / 10

Total: **100/100** samples byte-identical across four datasets, plus a further 10/10 on DSEC tested separately below.

DAVIS Camera Pose could not be tested: its loader fails with a FileNotFoundError (missing events.txt after download) -- a pre-existing bug in that dataset's loading path, unrelated to this work, and out of scope to fix here.

6b. Speed -- tonic original vs. numba candidate

Dataset	Events/sample	Original	Numba	Speedup
N-MNIST	4,059	0.70 ms	0.06 ms	12.4x
DVS128 Gesture	351,782	49.91 ms	5.64 ms	8.9x
N-Caltech101	133,469	22.30 ms	3.41 ms	6.5x
Eye Tracking	1,625	0.60 ms	0.80 ms	0.8x (SLOWER)
DSEC (thun_00_a)*	958,390	171.03 ms	25.46 ms	6.7x

The DVS128 Gesture figure (49.91 ms) closely matches the ~40-66 ms range measured earlier in the GIL probe, confirming ToFrame really is the cost being targeted. DSEC -- the dataset that motivated this entire investigation -- shows both the largest per-sample cost by far (171 ms, driven by ~958,000 events per sample at 640×480 resolution) and a meaningful absolute improvement (145 ms saved per sample) even though its relative speedup (6.7x) is the smallest positive result among the datasets tested.

Eye Tracking is a genuine exception, reported honestly rather than omitted: with only ~1,625 events per sample -- far fewer than every other dataset tested -- the numba candidate is actually *slower* than tonic's original (0.80 ms vs. 0.60 ms). This is a well-known JIT tradeoff, not a correctness problem (Eye Tracking still passed all 25/25 byte-identical checks): a compiled function's call/dispatch overhead is roughly fixed regardless of input size, so at a small enough event count that fixed overhead can exceed the actual work

saved. The practical implication is that this acceleration should be applied per-dataset based on typical event volume, not unconditionally to every dataset the pipeline supports.

** DSEC was tested against a single recording (thun_00_a, ~296 MB downloaded: events + optical-flow target only) rather than the full 18-recording optical-flow training set the production pipeline uses for real training, which is undocumented in size and estimated at "likely tens of GB." This keeps the validation bounded while still exercising DSEC's real 640x480 resolution and genuine per-sample event volume.*

7. Supplementary Findings From the Same Investigation

7a. The nogil=True fix: free to make, but not the lever that helps

Adding `nogil=True` to the denoise kernel is free and zero-risk -- already verified safe and GIL-releasing. But it has no effect on the pipeline as it exists today: the architecture already uses process-based workers, not threads, and each worker process has its own interpreter, so multiprocessing already fully sidesteps the GIL for both denoise and ToFrame right now. `nogil` would only matter if something switched to thread-based workers later -- which this investigation already ruled out, since ToFrame (the bigger cost) stays GIL-bound regardless.

7b. Throughput, not latency, is what keeps the GPU fed

Even at batch_size=1 with a slow per-sample cost, the real requirement is not "prepare this one sample faster" -- it is "keep a steady stream of finished samples flowing." That is exactly what the pipeline's existing worker-count and prefetch-depth calibration already targets: enough workers preparing future samples concurrently means the GPU is never left waiting, regardless of how long any single sample takes to prepare. This required no new code -- only confirming the calibrated prefetch depth is generous enough for DSEC's slower per-sample cost specifically, rather than assuming it is.

7c. What PyTorch's num_workers actually parallelizes

A common assumption was that a batch of size 128 with 4 workers means each worker prepares 32 items of the *same* batch. That is not how PyTorch's DataLoader works. Its default `BatchSampler` groups indices into whole batches *before* dispatching to workers, then hands each entire batch to one worker, round-robin. With 4 workers: worker 1 prepares all of batch 0, worker 2 prepares all of batch 1 (simultaneously), and so on -- parallel *across* batches, never a split *within* one batch. For DSEC's batch_size=1, this means more workers prepare more *upcoming* single-sample batches in parallel, which helps the pipeline stay ahead of the GPU -- but does not make any one sample's own preprocessing finish faster. That specific latency can only be reduced by making the preprocessing computation itself cheaper, which is exactly what this case study's numba acceleration of ToFrame achieves.

8. Conclusion

The investigation started from a plausible-sounding architectural question -- replace process workers with threads -- and the evidence ruled it out cleanly: the dominant preprocessing cost (ToFrame) cannot be parallelized under a thread model no matter what else changes, because it does not release Python's GIL and was never going to. Rather than force a threading solution onto a workload that structurally cannot benefit from it, the investigation redirected toward the actual, measured bottleneck and applied a proven, low-risk technique already established in this codebase: JIT-compile the expensive operation itself.

The result is a numba-accelerated `ToFrame` replacement that is byte-identical to the reference implementation on every dataset it could be tested against, and 6.7x to 12.4x faster -- including a 145 ms-per-sample absolute reduction on DSEC, the highest-resolution and most memory-constrained dataset this pipeline supports, which is precisely where the reduction matters most.

Four things are recommended going forward: (1) wire `FastToFrame` into the production pipeline (`event_data_workflow/data_pipeline.py`) for datasets with substantial per-sample event counts -- every dataset tested here except Eye Tracking -- mirroring how `FastDenoise` is already wired in; (2) leave Eye Tracking on tonic's original ToFrame, since the measured evidence shows the numba version is slower for it specifically; (3) leave the existing process-based worker architecture and its live RAM/VRAM-driven calibration untouched -- it is already the correct mechanism for keeping the GPU fed, and no thread-based alternative was found to improve on it; (4) treat the DAVIS Camera Pose loader's download/extraction failure as a separate, pre-existing issue worth fixing on its own, since it currently blocks any diagnostic work -- including this one -- from validating against that dataset at all.

9. Diagnostic Tooling Used to Produce These Results

The three scripts below produced every number in Section 6. They are documented here in full -- methodology and complete code -- as the permanent record of how validation was done, since the scripts themselves are one-off investigation tools, not part of the ongoing pipeline, and have been removed from the repository after this report was generated.

9a. validate_fast_to_frame.py -- correctness gate

Runs tonic's original `to_frame_numpy` against the numba candidate on real event samples pulled from the actual training pipeline (via the same `NeuromorphicEncoder`/dataset-registry path production training uses), for every registered dataset, and asserts byte-identical shape, dtype, and values for each sample. A per-dataset loader failure (as happened with DAVIS Camera Pose) is caught and reported as skipped rather than aborting the whole run. This is the gate that had to pass before the speed numbers in Section 6b were trusted at all.

```
"""
Correctness gate for event_data_workflow/fast_to_frame.py -- runs tonic's original
to_frame_numpy (n_time_bins mode) against the numba candidate on the same real event
samples (N-MNIST, DVS128 Gesture -- both already cached locally under tmp/data, no
download needed) and asserts byte-identical output (same shape, same dtype, same
values) for each. Mirrors diagnostics/validate_fast_denoise.py's structure and pass bar.

Only after this passes does a benchmark's speed numbers mean anything -- same rule as
fast_denoise.py: a fast function that produces wrong training data is worse than the
slow one it replaces.

Run from the project root: python diagnostics/validate_fast_to_frame.py
"""
import sys
import random
from pathlib import Path
PROJECT_ROOT = Path(__file__).resolve().parent.parent
sys.path.insert(0, str(PROJECT_ROOT))

import numpy as np
from tonic.functional import to_frame_numpy
from event_data_workflow.fast_to_frame import to_frame_numba
from skeleton import Settings
from event_data_workflow import NeuromorphicEncoder, resolve_dataset_entry

N_TIME_BINS = 16   # data_workflow.yaml's shipped default
N_SAMPLES_PER_DATASET = 25


def unwrap_raw_dataset(ds):
    """Peels every '.dataset' wrapper (cache classes, PreTransformedDataset, and for
    loader-based entries a torch Subset from random_split) down to the innermost real
    dataset -- depth differs between cls-based entries (N-MNIST/N-Caltech101/DVS Gesture)
    and loader-based ones (DAVIS Pose/DSEC/Eye Tracking), so a fixed number of .dataset
    hops isn't reusable across both; this just peels until there's nothing left to peel."""
    while hasattr(ds, "dataset"):
        ds = ds.dataset
    return ds


def check_dataset(dataset_name: str, n_samples: int) -> tuple[int, int]:
    cfg = Settings()
    cfg.DEVICE = "cuda"
    cfg.DATASET_NAME = dataset_name
    cfg.CALIBRATE_BATCH_SIZE = False
    cfg.BATCH_SIZE = 128
    entry = resolve_dataset_entry(cfg)
```

```
    cfg.DATASET_NAME = entry["name"]
    sensor_size = entry["sensor_size"]

    encoder = NeuromorphicEncoder(cfg)
    train_loader, _ = encoder.get_dataloaders()
    raw_dataset = unwrap_raw_dataset(train_loader.loader.dataset)

    indices = random.sample(range(len(raw_dataset)), min(n_samples, len(raw_dataset)))
    passed, failed = 0, 0
    for idx in indices:
        events, target = raw_dataset[idx]
        if len(events) == 0:
            continue  # empty-input zero-fill branch is FastToFrame's job, not to_frame_numba's -- nothing to compare he

        original = to_frame_numpy(events, sensor_size=sensor_size, n_time_bins=N_TIME_BINS)
        candidate = to_frame_numba(events, sensor_size=sensor_size, n_time_bins=N_TIME_BINS)

        same_shape = original.shape == candidate.shape
        same_dtype = original.dtype == candidate.dtype
        same_content = same_shape and np.array_equal(original, candidate)
        if same_content and same_dtype:
            passed += 1
        else:
            failed += 1
            print(f"  [MISMATCH] {dataset_name} idx={idx}: "
                  f"original shape={original.shape} dtype={original.dtype}, "
                  f"candidate shape={candidate.shape} dtype={candidate.dtype}")
            if same_shape and not same_content:
                diff = original != candidate
                first_bad = np.argwhere(diff)[0]
                t_i, p_i, y_i, x_i = first_bad
                print(f"    first diff at [T={t_i},P={p_i},Y={y_i},X={x_i}]: "
                      f"original={original[t_i, p_i, y_i, x_i]} candidate={candidate[t_i, p_i, y_i, x_i]}")
    return passed, failed


def main():
    random.seed(1234)
    all_ok = True
    total_passed, total_failed = 0, 0
    skipped = []
    for dataset_name in ["N-MNIST", "DVS128 Gesture", "N-Caltech101", "DAVIS Camera Pose", "Eye Tracking"]:
        print(f"[VALIDATE] {dataset_name}: comparing {N_SAMPLES_PER_DATASET} real samples "
              f"against tonic's original to_frame_numpy (n_time_bins={N_TIME_BINS})...", flush=True)
        try:
            p, f = check_dataset(dataset_name, N_SAMPLES_PER_DATASET)
        except Exception as e:
            print(f"  [SKIPPED] {dataset_name}: loader raised {type(e).__name__}: {e} "
                  "-- a dataset-loading issue, unrelated to fast_to_frame.py")
            skipped.append(dataset_name)
            continue
        print(f"  {dataset_name}: {p}/{p + f} samples byte-identical", flush=True)
        total_passed += p
        total_failed += f

    print(f"\n[VALIDATE] TOTAL: {total_passed}/{total_passed + total_failed} samples byte-identical")
    if skipped:
        print(f"[VALIDATE] SKIPPED (dataset-loading issues, not fast_to_frame.py): {skipped}")
    if total_failed == 0:
        print("[VALIDATE] PASS -- verified correct on real data.")
    else:
        print("[VALIDATE] FAIL -- do not use to_frame_numba until fixed.")
        all_ok = False

    if not all_ok:
        sys.exit(1)


if __name__ == "__main__":
    main()
```

diagnostics/validate_fast_to_frame.py -- full code (removed from the repository after this report).

9b. benchmark_fast_to_frame.py -- speed comparison

Times both implementations on the same real samples used for validation, after a single untimed warm-up call absorbs numba's one-time JIT compilation cost so it isn't charged to the first timed sample. Writes per-sample timing to a CSV and prints the per-dataset averages shown in Section 6b.

```
"""
Speed comparison: tonic's original to_frame_numpy (n_time_bins mode) vs.
event_data_workflow/fast_to_frame.py's numba candidate -- only meaningful because
diagnostics/validate_fast_to_frame.py already confirmed byte-identical output on real
data. Mirrors diagnostics/benchmark_fast_denoise.py's structure and warm-up rationale.

Run from the project root: python diagnostics/benchmark_fast_to_frame.py
(N-MNIST and DVS128 Gesture must already be downloaded -- both already cached locally
under tmp/data in this repo, no fresh download needed.)
"""
import sys
import csv
import random
import time
from pathlib import Path
PROJECT_ROOT = Path(__file__).resolve().parent.parent
sys.path.insert(0, str(PROJECT_ROOT))

from tonic.functional import to_frame_numpy
from event_data_workflow.fast_to_frame import to_frame_numba
from skeleton import Settings
from event_data_workflow import NeuromorphicEncoder, resolve_dataset_entry

DIAG = Path(__file__).resolve().parent
OUT_CSV = DIAG / "benchmark_fast_to_frame_results.csv"
N_SAMPLES_PER_DATASET = 40
N_TIME_BINS = 16


def unwrap_raw_dataset(ds):
    """See validate_fast_to_frame.py's copy of this helper for why a fixed number of
    .dataset hops isn't reusable across cls-based and loader-based dataset entries."""
    while hasattr(ds, "dataset"):
        ds = ds.dataset
    return ds


def benchmark_dataset(dataset_name: str, seed: int) -> list[dict]:
    cfg = Settings()
    cfg.DEVICE = "cuda"
    cfg.DATASET_NAME = dataset_name
    cfg.CALIBRATE_BATCH_SIZE = False
    cfg.BATCH_SIZE = 128
    entry = resolve_dataset_entry(cfg)
    cfg.DATASET_NAME = entry["name"]
    sensor_size = entry["sensor_size"]

    encoder = NeuromorphicEncoder(cfg)
    train_loader, _ = encoder.get_dataloaders()
    raw_dataset = unwrap_raw_dataset(train_loader.loader.dataset)

    random.seed(seed)
    non_empty = [i for i in range(len(raw_dataset)) if len(raw_dataset[i][0]) > 0]
    indices = random.sample(non_empty, min(N_SAMPLES_PER_DATASET, len(non_empty)))

    # Warm up numba's JIT compilation once, untimed, on a sample not in the timed set.
    warmup_events, _ = raw_dataset[indices[0]]
    to_frame_numba(warmup_events, sensor_size=sensor_size, n_time_bins=N_TIME_BINS)

    rows = []
    for idx in indices:
        events, _ = raw_dataset[idx]
```

```
        n_events = len(events)

        t0 = time.perf_counter()
        original_result = to_frame_numpy(events, sensor_size=sensor_size, n_time_bins=N_TIME_BINS)
        original_ms = (time.perf_counter() - t0) * 1000

        t0 = time.perf_counter()
        numba_result = to_frame_numba(events, sensor_size=sensor_size, n_time_bins=N_TIME_BINS)
        numba_ms = (time.perf_counter() - t0) * 1000

        assert original_result.shape == numba_result.shape, \
            f"idx={idx}: output mismatch during benchmark, re-run validate_fast_to_frame.py"
        rows.append({"dataset": dataset_name, "index": idx, "n_events": n_events,
                     "original_ms": round(original_ms, 3), "numba_ms": round(numba_ms, 3),
                     "numba_speedup": round(original_ms / numba_ms, 2)})
    return rows


DATASETS = ["N-MNIST", "DVS128 Gesture", "N-Caltech101", "DAVIS Camera Pose", "Eye Tracking"]


def main():
    all_rows = []
    skipped = []
    for i, dataset_name in enumerate(DATASETS):
        try:
            all_rows.extend(benchmark_dataset(dataset_name, seed=99 + i))
        except Exception as e:
            print(f"[SKIPPED] {dataset_name}: loader raised {type(e).__name__}: {e} "
                  "-- a dataset-loading issue, unrelated to fast_to_frame.py")
            skipped.append(dataset_name)

    with open(OUT_CSV, "w", newline="") as f:
        writer = csv.DictWriter(f, fieldnames=list(all_rows[0].keys()))
        writer.writeheader()
        writer.writerows(all_rows)

    print(f"\n[BENCHMARK] wrote {OUT_CSV}\n")
    if skipped:
        print(f"[BENCHMARK] SKIPPED (dataset-loading issues, not fast_to_frame.py): {skipped}\n")
    for dataset_name in [d for d in DATASETS if d not in skipped]:
        rows = [r for r in all_rows if r["dataset"] == dataset_name]
        avg_orig = sum(r["original_ms"] for r in rows) / len(rows)
        avg_numba = sum(r["numba_ms"] for r in rows) / len(rows)
        avg_events = sum(r["n_events"] for r in rows) / len(rows)
        print(f"[BENCHMARK] {dataset_name}, {len(rows)} real samples, avg {avg_events:,.0f} events/sample")
        print(f"  original (tonic, per-slice np.add.at) : {avg_orig:.2f}ms/sample")
        print(f"  numba (JIT, same algorithm)           : {avg_numba:.2f}ms/sample  ({avg_orig/avg_numba:.1f}x)")


if __name__ == "__main__":
    main()
```

diagnostics/benchmark_fast_to_frame.py -- full code (removed from the repository after this report).

9c. dsec_subset_check.py -- bounded DSEC validation

DSEC's production loader (`load_dsec()`) fetches all 18 optical-flow recordings, an undocumented, "likely tens of GB" download. This script instead constructs tonic's DSEC class directly with a single named recording, keeping the download to what one recording's events plus its optical-flow target actually costs (~296 MB for the recording used here) -- without altering the production loader other code depends on. It then windows that one recording's raw event stream by the real optical-flow target timestamps, the same slicing logic `WindowedRecordingDataset` applies for real training samples, so the per-sample event count tested here matches what a real training run would actually see.

```
"""
DSEC ToFrame check on ONE recording only, not the full 18-recording optical-flow set
load_dsec() (event_data_workflow/dataset_registry.py) fetches for real training --
that set is undocumented in size and "likely tens of GB" per the registry's own
comment. This uses tonic's DSEC class directly with a single recording name in
`split`, keeping the download to what one recording's events_left + optical-flow
target actually costs, without touching the production load_dsec()/DSEC_RECORDINGS
definition other code depends on.

Windows the recording the same way event_data_workflow/dataset_registry.py's
WindowedRecordingDataset does for real training samples -- events sliced by each
optical-flow target's (start_us, stop_us) pair -- rather than testing on one huge
whole-recording event array, so the per-sample event count here matches what a real
training run would actually see.

Run from the project root: python diagnostics/dsec_subset_check.py
"""
import sys
import time
from pathlib import Path
PROJECT_ROOT = Path(__file__).resolve().parent.parent
sys.path.insert(0, str(PROJECT_ROOT))

import numpy as np
import tonic
from tonic.functional import to_frame_numpy
from event_data_workflow.fast_to_frame import to_frame_numba

DATA_DIR = PROJECT_ROOT / "tmp" / "data"
RECORDING = "thun_00_a"   # one recording only -- not the full 18-recording DSEC_RECORDINGS set
N_TIME_BINS = 16
N_SAMPLES = 10


def main():
    print(f"[DSEC] downloading ONLY recording '{RECORDING}' (events_left + optical flow target) -> {DATA_DIR}")
    dsec = tonic.datasets.DSEC(
        save_to=str(DATA_DIR), split=[RECORDING], data_selection="events_left",
        target_selection=["optical_flow_forward_event", "optical_flow_forward_timestamps"],
    )
    sensor_size = tonic.datasets.DSEC.sensor_size

    data_tuple, target_tuple = dsec[0]
    events = data_tuple[0]["events_left"]
    _, timestamps = target_tuple   # (optical_flow_forward_event, optical_flow_forward_timestamps)
    print(f"[DSEC] '{RECORDING}': {len(events):,} total events, {len(timestamps)} optical-flow windows, sensor {sensor_s

    n = min(N_SAMPLES, len(timestamps))
    windows = [events[(events["t"] >= start) & (events["t"] < stop)] for start, stop in timestamps[:n]]
    windows = [w for w in windows if len(w) > 0]

    passed, failed = 0, 0
    for i, window in enumerate(windows):
        original = to_frame_numpy(window, sensor_size=sensor_size, n_time_bins=N_TIME_BINS)
        candidate = to_frame_numba(window, sensor_size=sensor_size, n_time_bins=N_TIME_BINS)
```

```
        if original.shape == candidate.shape and original.dtype == candidate.dtype and np.array_equal(original, candidat
            passed += 1
        else:
            failed += 1
            print(f"  [MISMATCH] window={i}: original shape={original.shape} candidate shape={candidate.shape}")
    print(f"\n[DSEC] correctness: {passed}/{passed + failed} windows byte-identical")

    to_frame_numba(windows[0], sensor_size=sensor_size, n_time_bins=N_TIME_BINS)  # JIT warm-up

    orig_times, numba_times, event_counts = [], [], []
    for window in windows:
        event_counts.append(len(window))
        t0 = time.perf_counter()
        to_frame_numpy(window, sensor_size=sensor_size, n_time_bins=N_TIME_BINS)
        orig_times.append((time.perf_counter() - t0) * 1000)
        t0 = time.perf_counter()
        to_frame_numba(window, sensor_size=sensor_size, n_time_bins=N_TIME_BINS)
        numba_times.append((time.perf_counter() - t0) * 1000)

    avg_orig = sum(orig_times) / len(orig_times)
    avg_numba = sum(numba_times) / len(numba_times)
    avg_events = sum(event_counts) / len(event_counts)
    print(f"[DSEC] {RECORDING}, {len(orig_times)} windows, avg {avg_events:,.0f} events/window")
    print(f"  original (tonic, per-slice np.add.at) : {avg_orig:.2f}ms/sample")
    print(f"  numba (JIT, same algorithm)           : {avg_numba:.2f}ms/sample  ({avg_orig/avg_numba:.1f}x)")


if __name__ == "__main__":
    main()
```

diagnostics/dsec_subset_check.py -- full code (removed from the repository after this report).